

RELATÓRIO 09 — MODO ANTHROPIC NO ROUTER

M.I.N.D. — Muñdji Intelligent Neural Developer
Quinto modo de serviço: ligação directa à API da Anthropic

Campo	Valor
Data	5 de Agosto de 2026
Commit	a71e5f8
Ficheiros tocados	5 — model_router.py, test_model_router.py, conftest.py, .env.example, docs/decisoões/gpu_alugada.md
Suite	341 passam · 4 saltados (eram 329 + 12 novos)
Âmbito	Só o router e os testes, como a especificação manda. Nenhum componente do sistema mudou.
Consequência prática	O piloto deixa de depender de alugar uma GPU. Falta uma chave de API, não uma máquina.

1. O que fiz

O *model_router* já suportava quatro modos de servir modelos (*openai_compat*, *hf_local*, *hf_api*, *ollama*). Acrescentei o quinto: **anthropic** — ligação directa à API da Anthropic, sem proxy intermédio. O padrão já existia; era mais um caso no despacho.

Antes de escrever uma linha confirmei a forma do pedido e o identificador de modelo sugerido pela especificação contra a referência oficial da API. O *claude-sonnet-4-6* que o documento dá como exemplo é um identificador válido e activo, e a forma HTTP descrita coincide exactamente com a documentada. Não havia nada a corrigir na especificação — mas verificar antes custa menos do que descobrir um 400 com a GPU a contar tempo.

Porque HTTP directo e não o SDK da Anthropic: o *model_router* é, por desenho, neutro em relação ao fornecedor — os quatro modos existentes falam todos *httpx*, e a especificação deu explicitamente a forma HTTP. Introduzir uma dependência de SDK só para um dos cinco modos partiria a simetria do ficheiro sem ganho nenhum. A entrega é aditiva: um fornecedor a mais ao lado dos outros.

2. Como fiz — as quatro diferenças, e porque cada uma tem teste

O modo *anthropic* parece o *openai_compat* e não é. Difere em quatro pontos, e **nenhum deles falha de forma barulhenta**: ou devolve 400, ou lê-se uma resposta vazia sem saber porquê. Por isso cada um tem o seu teste — um teste único que só verificasse «devolve texto» passaria com três dos quatro errados.

Ponto	openai_compat	anthropic	Teste
Autenticação	Authorization: Bearer <k>	x-api-key: <k>	<i>autentica_com_x_api_key</i> — e confirma que <i>Authorization</i> não vai
Versão da API	não existe	anthropic-version: 2023-06-01 obrigatório	<i>envia_o_header_de_versao</i>
System prompt	mensagem de <i>role: system</i> dentro de <i>messages</i>	campo <i>system</i> de topo, fora de <i>messages</i>	<i>poe_o_system_no_campo_proprio</i> — verifica os dois lados

Ponto	openai_compat	anthropic	Teste
Leitura da resposta	choices[0].message.content	content[0].text	le_a_resposta_de_content_text
max_tokens	opcional	obrigatório (4096, valor da especificação)	envia_max_tokens

Mais quatro testes fecham o resto: montagem do URL (*/v1/messages*), omissão do endpoint, precedência de um endpoint explícito, e retry em 529. Os 4 modos anteriores continuam a passar sem uma única alteração — a parametrização sobre *MODOS* passou a cobrir cinco automaticamente. **341 testes, 0 falhas.**

3. Três decisões que tomei, e a razão de cada uma

a) A resposta não é lida cegamente pelo índice 0. A especificação diz *content[0].text*. Implementei a procura do primeiro bloco de tipo «text». Não é uma liberdade: a lista *content* é heterogênea por desenho, e se algum dia vier outro tipo de bloco à frente, *content[0]* não tem sequer a chave «text» — seria um *KeyError* a meio de um ciclo em vez de uma resposta. O teste da especificação passa na mesma; há um teste extra para o caso heterogêneo.

b) A omissão do endpoint saiu para uma função própria. *MODEL_ENDPOINT* passa a ter omissão de *https://api.anthropic.com* quando o modo é *anthropic*, como pedido. Pus a regra em *endpoint_por_omissao()* em vez de a espalhar pelos três ramos do *component_config*. Um *MODEL_ENDPOINT* explícito continua a mandar em **qualquer** modo — é o que permite apontar o modo *anthropic* a um proxy compatível sem inventar um sexto modo. Há teste de regressão a garantir que os outros modos mantêm *http://localhost:8000*.

c) O conftest passou a limpar MODEL_ENDPOINT. Faltava na lista de variáveis isoladas. Enquanto nenhum teste dependia da omissão isso não se notava; o teste novo depende, e uma *.env* do utilizador com *MODEL_ENDPOINT* preenchido fá-lo-ia falhar por razão errada. É uma falha de isolamento que já lá estava — o teste novo só a tornou visível.

4. O que NÃO fiz, e porquê

A especificação foi explícita: «O QUE NÃO MUDA: nada mais. Esta entrega é só no router e nos testes.» Respeitei-o. O que se segue são coisas que a API da Anthropic oferece e que **deliberadamente ficaram de fora**:

Não fiz	Porquê
<i>max_tokens</i> configurável	A especificação fixa 4096. Deixei-o em constante de módulo (<i>ANTHROPIC_MAX_TOKENS</i>) — muda-se numa linha quando houver decisão. Mas ver o risco em baixo : não é um detalhe cosmético.
<i>streaming</i>	Todo o router é síncrono e devolve texto completo. Introduzir streaming num modo só partiria a promessa de que <i>generate()</i> funciona identicamente em todos.
<i>prompt caching</i>	Reduziria custo real (as personas e as instruções JSON repetem-se em todas as chamadas). Fora de âmbito, e sem medições de custo não há como provar o ganho. Fica na lista do que se pode fazer.
<i>extended thinking</i>	Mudaria a qualidade das decisões do CORTEX e do CEREBELLUM — e por isso mudaria o que o piloto mede. Ligar isto antes de medir a base seria estragar a linha de referência.
Tocar em CORTEX / CEREBELLUM / NEURONS	Não precisam de saber. Chamam <i>router.generate()</i> e recebem texto. É essa transparência que torna o quinto modo uma entrega de cinco ficheiros em vez de trinta.

Risco que fica em aberto: truncagem silenciosa a 4096 tokens

Reporto isto antes de corrigir, e não corriji. Nos outros modos *max_tokens* é opcional e o servidor decide. Neste é obrigatório e ficou em 4096. Um NEURON a gerar um ficheiro grande pode bater no limite, e a API devolve o texto **truncado** com *stop_reason: max_tokens* — não devolve erro. O router não lê o *stop_reason*, portanto o código truncado seguiria para a sandbox como se estivesse completo, falharia a compilar, e o CEREBELLUM reprovaria a iteração **pela razão errada**.

Não mexi porque 4096 é o valor da especificação e mudá-lo por iniciativa própria é exactamente o tipo de ajuste às cegas que ficou proibido. Mas é a primeira coisa a observar no piloto: se aparecerem reprovações com código cortado a meio, a causa é esta e não o modelo. A correcção certa é ler o *stop_reason* e reportá-lo — não subir o número até deixar de doer.

E continua a não haver piloto com modelos reais. Estes 12 testes provam que o router *formata* o pedido como a API espera. Provam-no contra um *httpx.post* substituído — nenhum byte saiu para a Anthropic. Que a API responde como o teste finge é a hipótese, não o resultado.

5. O que isto muda no estado do projecto

Mais do que parece para cinco ficheiros. Até hoje o piloto estava bloqueado em **três** valores que exigiam alugar uma máquina: endpoint, token e identificador de modelo — mais o tempo de aprovisionar o vLLM e de o manter de pé. Com o modo anthropic, o endpoint tem omissão, o modelo é um identificador público, e o que falta é **um valor**:

Antes (GPU alugada)	Agora (modo anthropic)
Alugar instância, aprovisionar vLLM, abrir túnel, escolher e descarregar modelos, manter a máquina de pé durante o piloto	MODEL_MODE=anthropic MODEL_AUTH_TOKEN=<chave> CORTEX_MODEL=<id>
Custo por hora de máquina, independente de se usar	Custo por token efectivamente gasto
Escolha de modelos é uma pré-condição do piloto	Um identificador público chega para arrancar

Isto **não** torna a GPU alugada obsoleta nem revoga nada do que ficou decidido. Os modos convivem, e a decisão de *docs/decisoes/gpu_alugada.md* mantém-se para modelos retreinados e adaptadores LoRA — que a API da Anthropic não serve. O que muda é que **já não é preciso resolver a escolha de modelos para poder medir o sistema**. Mede-se primeiro, escolhe-se depois — que é, aliás, a ordem que ficou escrita desde o início.

A regra sobre a SYNAPSE DB mantém-se intacta: os prompts saem, a base de dados não. O modo anthropic tem exactamente a mesma natureza da GPU alugada nesse ponto — inferência remota, dados locais — e documentei-o assim no *docstring* e na tabela de decisões, sem fingir que é mais privado do que é.

6. O que poderia fazer

- **Ler o *stop_reason* e propagá-lo.** Fecha o risco da secção 4 de forma honesta: uma resposta truncada passa a ser reportada como truncada, em vez de ser confundida com código mau.
- **Prompt caching nas personas e nas instruções JSON.** São a parte constante de todas as chamadas e repetem-se dezenas de vezes por ciclo. Com o custo a ser por token, é o ganho mais directo que existe.
- **Registar tokens gastos por componente na SYNAPSE DB.** A resposta traz *usage*. Guardá-lo daria custo real por ciclo e por componente — exactamente o tipo de número que o critério de selecção de modelos precisa e que hoje não existe.
- **Modos diferentes por componente.** Hoje *MODEL_MODE* é global. Poder correr o CORTEX em anthropic e os NEURONS numa GPU alugada é arquitecturalmente trivial (o token já é por componente) e abre a hipótese de misturar por custo.
- **Extended thinking no CORTEX e no CEREBELLUM.** São os dois componentes que decidem. Mas só depois de haver uma linha de referência sem ele — caso contrário não se sabe o que melhorou.

7. O que vou fazer a seguir

#	Passo	Depende de
1	Correr o piloto real em modo anthropic. As quatro passagens do runbook, agora com um modelo verdadeiro do outro lado. É o que está por provar desde o primeiro dia.	Uma chave de API. Mais nada.
2	Observar especificamente a truncagem a 4096 e a conformidade com o esquema JSON — os dois números que só se medem com modelo real.	Passo 1
3	Consoante o que sair: ou fechar o <i>stop_reason</i> , ou registar o <i>usage</i> , ou ambos. Pela ordem que os resultados ditarem , não por esta lista.	Passo 2
4	Só então preencher <i>config/neuron_specialties.yaml</i> . Continua a valer a regra: as especialidades saem dos resultados medidos, nunca de estimativa.	Passo 2

O passo 1 deixou de estar bloqueado em infraestrutura. Está bloqueado num valor de configuração — e é uma decisão tua, não uma dependência técnica.